Cheatsheet AZ-305 — 05 · OAuth, application identities, Key Vault and API Management

1. Roles in OAuth 2.0 with Microsoft Entra ID

Role	Who	What it does
Authorization Server	Microsoft Entra ID (or B2C)	**Authenticates, asks for consent and issues/signs the tokens**
Client	The calling app (web app, SPA, service)	Requests the token and sends it in Authorization: Bearer
Resource Server	The API	**Validates** the token (signature, aud , expiry, claims) and decides **authorization**
Resource Owner	The user	Gives consent

- "Access tokens generated by" → **Entra ID**. "Authorization decisions performed by" → **the Web API**.
- Key Vault never issues OAuth tokens.

2. Grant types

Grant	User?	Client secret	Use	Status
Authorization Code (+PKCE)	Yes	Yes (PKCE if it cannot store it)	Web apps, mobile, SPAs	✓ Recommended
Client Credentials	**No**	Yes (secret / certificate)	App-to-app, jobs, backends, managed identities	✓ For M2M
Implicit	Yes	No	(Legacy) SPAs	✗ Deprecated
Resource Owner Password	Yes (the app sees the password)	Yes	(Legacy) migrations	✗ Avoid; incompatible with MFA
Device Code	Yes	No	Devices without a browser	Niche

- **state parameter:** opaque value that goes out and comes back in the redirect; CSRF protection and **transport of the app's own data ("custom data")**. Only makes sense with a redirect (Auth Code / Implicit).

3. Delegated vs Application permissions

	Delegated	Application
Acts on behalf of	**A signed-in user**	**The app** (no user)
Flow	Authorization Code	Client Credentials / Managed Identity
Consent	User or admin	**Always admin consent**
Claim in the token	scp (scopes)	roles (app roles)
Effective scope	What the user can do	Everything granted, across the whole organization
Defined in the API as	Scope (Expose an API)	App role (App roles, member type Applications)

4. App registration: which blade each app touches

Need	App	Blade
Expose permissions/roles to other apps	The **API**	**Expose an API** (scopes) / **App roles**
Request permissions/roles of another API (so they appear in the token)	The **client**	**API permissions** (+ admin consent)
Add optional claims to the token	Any	Token configuration
Register an app for Entra ID authentication	The app	App registration (the basis of everything)
Publish an on-prem app with SSO without VPN	—	Application Proxy + Enterprise Application (KCD for Kerberos)
App registration vs Enterprise application	Registration = definition (one); Enterprise app = instance/service principal in the tenant (allows assigning users, CA, SAML SSO)	

- Silent SSO on **Entra-joined** devices → app registration + the device's **PRT**; nothing else is needed.
- Access only from company computers → Conditional Access · Grant · compliant / joined device.

5. Identities for applications

Identity	Where the app runs	Credentials	When
Managed identity (system / user-assigned)	**Azure resources only** (VM, App Service, Functions, AKS, Policy remediation, Data Factory)	None to manage	Whenever the app is in Azure
Service principal	Anywhere (on-prem, another cloud, CI/CD)	Secret or certificate	App outside Azure; staff rotation
Personal access token / user	—	Tied to a person	Avoid (staff turnover)

Identity	Where the app runs	Credentials	When
Certificate in Key Vault	Apps that need mTLS or signing	Managed certificate	App authentication with cert

- **IMDS** (169.254.169.254/metadata/identity/oauth2/token): local endpoint through which a **VM** obtains the token of its managed identity. "Which endpoint to obtain an access token" with managed identity → IMDS.
- **User-assigned** MI: reusable across several resources, independent lifetime. **System-assigned**: created and deleted with the resource.

6. Key Vault: three objects

Object	What it is	Operation	Examples
Secret	Sensitive text retrieved as is	GET secret	Connection strings, passwords, API keys
Key	Cryptographic key (RSA/EC) that **never leaves**	encrypt / decrypt / sign / wrap	CMK for Storage / SQL TDE / ADE, signing
Certificate	X.509 certificate with its key and renewal	GET certificate , auto-renew	TLS, mTLS, app signing

- **Standard SKU** (software) vs **Premium** (**HSM-backed keys, FIPS 140-2 Level 2**) vs **Managed HSM** (Level 3, single-tenant).
- Access: **RBAC** (recommended) or access policies; the app's **managed identity** as caller; **Private Endpoint** for private access.
- "Protect connection strings, only the app at runtime, no hard-coded credentials" → **Key Vault** (the store), with managed identity as the access mechanism.

7. Encryption: where each technology sits

State	Technology
At rest	SSE (Storage, PMK/CMK), **TDE** (SQL, files), **Azure Disk Encryption** (BitLocker/DM-Crypt + customer Key Vault, auditable), Always Encrypted (columns; only the client decrypts)
In transit	TLS/HTTPS, IPsec (VPN), MACsec (ExpressRoute Direct)
In use	**Azure Confidential Computing** (TEE, Confidential VMs)
Masking by role	**Dynamic Data Masking** (some see everything, others the last 4 digits)

- mTLS in App Service: the client certificate arrives in the **X-ARR-ClientCert** header.
- P2S VPN with certificates: upload the **root public key** to Azure; client certificates stay on the devices; individual revocation by thumbprint.

8. API Management

Concept	What it is
Products	Packaging of APIs for consumers: subscriptions, quotas, terms
Policies	XML in inbound / backend / outbound / on-error: validate-jwt , rate-limit , quota , set-header , ip-filter , cors , validate-client-certificate , XML↔JSON transformation
VNet modes	**External**: gateway with public IP + private backends · **Internal**: everything private · **None**: no VNet
Tiers	Consumption (serverless) · Developer (not for production) · Basic · Standard · **Premium** (VNet inject, multi-region, ISE-like) · v2 tiers (Basic v2, Standard v2 with outbound VNet integration)
VNet injection (External/Internal)	Only **Developer and Premium** (classic)
Self-hosted gateway	APIM gateway deployed on-prem / other clouds

- "third-party OAuth provider" + different rate limits per consumer + "no changes to backend" + "no guest accounts" → **APIM** (validate-jwt + Products).
- "remove AspNet-Version header" → **policy** (set-header delete).
- "private IP + mTLS + rate limiting" (APIM options) → **Premium Internal** (exam answer; Standard does not inject into a VNet).
- Chain Front Door → APIM → backend: **OWASP → Front Door; JWT/ratelimit/products → APIM; IP filtering per-API → APIM** (in matching, B2C gets "JWT").
- Functions HTTP trigger: **Anonymous** (public) · **Function** (key for trusted apps) · **Admin** (master key). Read-only → **GET only**; CORS is handled by the platform (do not add OPTIONS).

9. Decision tree (app identity)

```
Where does the app run?
├─ In Azure ............................ Managed identity (token via IMDS on VMs)
├─ Outside Azure ....................... Service principal (secret/certificate in Key Vault)
└─ Does it act on behalf of a user?
     ├─ Yes → Authorization Code, delegated permissions (scp)
     └─ No → Client Credentials, application permissions (roles), admin consent
```

10. Signals → answer

Wording	Answer
"tokens generated by"	Microsoft Entra ID
"authorization decisions by"	The Web API
"custom data in the grant flow"	Support state parameter
"token must include the Writer role of App1" (modify App2)	API permissions of App2
"grant permissions using Entra-generated claims"	Microsoft Entra ID
"JWT validation policy using"	Azure API Management
"no user interaction", "service to service"	Client Credentials / application permission
"obtain access token" + managed identity on a VM	IMDS
"on-premises app" + "staff turnover"	Service principal
"retrieve keys for use in code" (Development team)	Managed identity
"connection strings", "runtime only"	Key Vault (secrets)
"FIPS 140-2 Level 2 HSM"	Key Vault Premium
"verify client certificate in web app"	HTTP request header
"data in use"	Confidential Computing
"supervisors see full number, operators last 4 digits"	Dynamic Data Masking
"never in plain text in the DB, only client decrypts"	Always Encrypted

11. Mistakes to avoid

1. Managed identity for an on-prem app → it does not exist outside Azure.
2. Managed identity as a secret "store" → it is the access mechanism; the store is Key Vault.
3. Confusing app registration (definition) with enterprise application (instance).
4. Implicit or ROPC as the "recommended" flow.
5. Modifying App roles in the client app → API permissions is what gets modified.
6. APIM Standard for a private IP → it does not inject into a VNet.